

# Glide – Aufgaben und Listen

Vollständige Projekt-, Produkt- und Wettbewerbsanalyse

Stand 16.09.2026

## Glide – Aufgaben und Listen

### Vollständige Projekt-, Produkt- und Wettbewerbsanalyse

**Stand der Analyse: 16.09.2026**

**Aktueller dokumentierter Produktstand: Glide 3.21.4**

**Aufgaben-/Datenformat: 15 · Einstellungen: 2 · Vorlagenformat: 2**

## 1. Gesamturteil

Glide ist im aktuellen Stand keine frühe To-do-App mehr, sondern ein bereits umfangreicher **lokaler Desktop-Aufgaben- und Planungsmanager** mit eigener Planungslogik, mehreren Ansichten desselben Datenbestands, Vorlagen- und Workflow-System, lokalen Anhängen, Erinnerungen, Kalenderimport und -export, Änderungsverlauf, robustem Backup-/Migrationskonzept und einer ungewöhnlich umfangreichen internen Qualitätssicherung.

Die funktionale Entwicklung ist dabei **weiter fortgeschritten als die eigentliche Produktveröffentlichung**.

Der vollständige Abschlusslauf für 3.21.4 wurde am 15.09.2026 auf macOS mit Python 3.14.5 erfolgreich durchgeführt: 39 Prüfschritte, davon 37 ausgeführt, alle 25 Testsuiten, drei Analysen sowie Beispiel- und Releasedatenabgleiche bestanden. Übersprungen wurden ausschließlich die plattformgebundene Screenshot-Erzeugung und die menschliche Sichtprüfung.

Damit sind die verbleibenden Hauptprobleme **nicht mehr fehlende Kernfunktionen**. Sie liegen inzwischen hauptsächlich bei:

- nativer Windows- und visueller Plattformabnahme,
- Packaging und Distribution,
- Signierung und Notarisierung,
- endgültiger Produkt-/Plattformidentität,
- Marken- und Lizenzentscheidung,
- Endnutzer-Onboarding,
- Langzeit-/Performance-Nachweisen,
- und der technischen Wartbarkeit des stark gewachsenen Anwendungskerns.

Glide befindet sich deshalb in einer Übergangsphase:

**vom umfangreich entwickelten und gut geprüften Quellprojekt zum installierbaren, nativ abgenommenen und vermarktaren Desktopprodukt.**

## 2. Was die zuletzt gefundenen Dateien zusätzlich verändern

Die drei zuletzt gefundenen Dokumente verändern den aktuellen Funktionsstand kaum, aber sie verbessern die **Entwicklungshistorie und Anforderungsnachverfolgung** deutlich.

### 2.1 Ursprünglicher Funktionsauftrag ist nachvollziehbar

25\_FEATURE\_ABGLEICH\_3.7.0.md dokumentiert den früheren Auftrag mit **22 einzelnen Anforderungen** und ordnet ihnen Umsetzung und Nachweise zu. Dazu gehören unter anderem Navigation, Farben, Listen-/Ordnerdialoge, lokale Schriftressourcen, Datenordner und Sperrdatei, Vorlagen, Export, Personalisierung, Jahresanzeige und Materialoptik.

Das ist relevant, weil Glide damit nicht als ungeplant gewachsene Ansammlung von Features einzuordnen ist. Ein erheblicher Teil der Oberfläche und Datenfunktionen entstand aus einem dokumentierten Anforderungskatalog.

Das historische Dokument trennt außerdem sauber zwischen tatsächlich umgesetzten Funktionen und bloß bewerteten Ideen. Frei wählbares Datumsformat, Kachelreihenfolge, Standardliste und Begrüßungston waren beispielsweise Erweiterungsideen; weitere analoge Anzeigeelemente wurden bewertet, aber ausdrücklich nicht umgesetzt.

## 2.2 Die Reiterfunktion hat eine klare Produktentscheidung

Der historische Reiterentwurf dokumentiert eine wichtige Entscheidung:

**Reiter werden vom Nutzer gezielt für vorhandene Punkte geöffnet. Gruppen werden nicht automatisch zu Reitern.**

Ein Reiter ist nur eine weitere Sicht auf dasselbe Objekt. Es entstehen keine Kopie, kein neuer Status und keine parallele Datenhaltung.

Das passt zu einem zentralen Architekturprinzip von Glide:

**Mehrere Ansichten – ein Datenobjekt.**

Auch offene Reiter gehören deshalb nicht zu den Aufgabendaten, sondern zum persönlichen Ansichtszustand in settings.json.

## 2.3 Die Dokumentationspflege ist historisch nachvollziehbar

Der zusätzliche Dokumentationsabgleich zeigt, dass ältere technische Fassungen nicht einfach überschrieben wurden. Überholte Dokumente wurden archiviert, historische Nachweise erhalten und aktuelle Einstiege auf einen kanonischen Dokumentationspfad umgestellt.

Das bestätigt die inzwischen erkennbare Dokumentationsstrategie:

**Aktuelle Dokumente werden fortgeschrieben. Historische Funktions-/Versionsverträge bleiben eingefrorene Nachweise.**

## 3. Produktdefinition

Die Projektdokumentation definiert Glide aktuell als:

eine deutschsprachige lokale Desktop-Anwendung für Aufgaben, Listen und Ordner, deren Kernfunktionen kein Internet, Benutzerkonto oder Cloudservice benötigen.

Diese Definition ist wichtiger als einzelne Features, weil daraus die Produktgrenzen folgen.

### Zielrichtung

Glide ist primär:

**persönliches Aufgabenmanagement + Tagesplanung + visuelle Organisation + lokale Datenhaltung.**

Es ist aktuell ausdrücklich **kein**:

- Team-Projektmanagementsystem,
- Cloudservice,
- kollaborativer Workspace,
- vollständiger Wissensmanager,
- Rich-Text-Notizeditor,
- Mail-Client,
- Kalender-Synchronisationsdienst,
- oder SaaS-Dienst mit Konto-/Lizenzserver.

Mehrbenutzerbetrieb, Konfliktzusammenführung, eigener Cloudservice, Telemetrie, Mehrsprachigkeit, externe Mail-/Kalenderintegration, Rich Text und Push bei geschlossener App sind aktuell dokumentierte Produktgrenzen.

Das ist bei der Konkurrenzanalyse wichtig:

**Nicht jede fehlende Konkurrenzfunktion ist automatisch ein Entwicklungsdefizit.**

## 4. Technischer Aufbau und Projektbestand

Der aktuelle Anwendungskern liegt in:

`src/glide/app.pyw`

Die Anwendung verwendet Python, Tk/Tcl und die Standardbibliothek. Die mitgelieferte DejaVu-Sans-Familie wird privat innerhalb des Prozesses registriert und nicht systemweit installiert.

### Projektbestand lässt sich in sieben Ebenen gliedern

#### Anwendung

- `app.pyw`
- Ressourcen
- Schriftdateien
- Vorlagenkatalog

#### Nutzerdaten

- Aufgabenbestand
- Einstellungen
- Vorlagen
- Anhänge
- Backups
- Lock-Datei
- Datenordner-Zeiger

#### Aktuelle technische Dokumentation

Unter anderem:

- Produktgrenzen
- Architektur
- Startkontext
- QA-Testplan
- Daten-/Backup-/Migrationsvertrag
- QA-Bericht
- Projektübergabe
- Release-Checkliste
- Dokumentationsindex

#### Versionierte Funktionsverträge

Von Erinnerungen über Reiter/Pinnwand, Tagesplanung, Backup und CSV bis zu ICS-Import/-Export.

#### Historisches Archiv

Ältere Bestandsanalysen, Abschlussberichte, Featureabgleiche und Versionsdokumente bleiben als Nachweise erhalten.

#### QA- und Analysewerkzeuge

Unter anderem:

- `pruefen.py`
- `analyse_statisch.py`
- `analyse_erreichbarkeit.py`
- `standpruefung.py`

- dauerlauf.py
- leistungspruefung.py
- screenshots.py
- symbolpruefung.py

### Reproduzierbare Projektbestände

- Beispielbestand
- Vorlagenbestand
- Release-Arbeitsbestand
- Referenz-Fixtures verschiedener Datenformate

Das Repository ist damit **deutlich strukturierter als der eigentliche Produktionscode**.

## 5. Daten- und Objektmodell

Glide arbeitet mit einem klaren hierarchischen Modell.

### Ordner

Ordner strukturieren **Listen und weitere Ordner**.

Maximal sind fünf Ordner Ebenen vorgesehen.

### Listen

Listen enthalten die eigentlichen Punkte.

### Aufgaben

Normale ausführbare Punkte mit Eigenschaften wie:

- erledigt,
- Wichtigkeit,
- Fälligkeit,
- Fälligkeitszeit,
- Wiederholung,
- Benachrichtigung,
- Bearbeitungstag,
- Aufwand,
- Labels,
- Farbe,
- Beschreibung,
- Anhänge.

### Long-Tasks

Long-Tasks sind mehrzeilige, notizähnliche Punkte, bleiben aber Teil des Aufgabenmodells.

### Gruppen

Gruppen organisieren Punkte innerhalb einer Liste.

Sie sind ein Container und selbst keine erledigbare Aufgabe.

### Überschriften

Überschriften sind reine visuelle Gliederungselemente.

Die interne Produktregel lässt sich daher sehr einfach ausdrücken:

**Aufgaben ordnet die Gruppe; Listen ordnet der Ordner.**

Diese semantische Trennung ist für das spätere Onboarding besonders wichtig.

## 6. Aktueller Funktionsumfang

### Aufgabenmanagement

Umgesetzt sind unter anderem:

- Aufgaben
- Unterpunkte
- Long-Tasks
- Gruppen
- Überschriften
- Fälligkeiten
- Uhrzeiten
- sechs Wiederholungsarten
- Wichtigkeiten
- Labels
- Farben
- Beschreibungen
- lokale Anhänge
- Mehrfachauswahl
- Drag-and-drop
- Suche
- Offen-Filter
- Rückgängig
- Papierkorb
- Änderungsverlauf

## 7. Planungssystem – eine der stärksten Produktideen

Besonders relevant ist die Trennung von drei verschiedenen Konzepten.

### Fälligkeit

Die Fälligkeit beantwortet:

**Bis wann muss etwas erledigt sein?**

Sie ist die eigentliche Deadline.

### Bearbeitungstag

Seit 3.14 besitzen Aufgaben und Long-Tasks zusätzlich einen freiwilligen `planned_date`.

Der Bearbeitungstag beantwortet:

**Wann möchte ich daran arbeiten?**

Ein Bearbeitungstag kann ausdrücklich ohne Fälligkeit existieren.

### Mein Tag

„Mein Tag“ ist wiederum eine **bewusst zusammengestellte Auswahl**.

Geplante Aufgaben werden nicht automatisch in „Mein Tag“ übernommen.

Damit existieren drei getrennte Ebenen:

**Deadline → Planung → persönlicher Fokus**

Das ist produktseitig wesentlich klarer als ein einziges universelles Datumsfeld.

## Geschätzter Aufwand

Seit 3.14 können Aufgaben zusätzlich einen Aufwand zwischen 1 und 60.000 Minuten erhalten.

## Tageskapazität

3.15 ergänzt daraus die Tagesplanung und eine persönliche Tageskapazität.

Die Bilanz wird aus dem aktuellen Bestand berechnet und nicht separat gespeichert.

Dadurch nähert sich Glide einem persönlichen Workload-Planner an, ohne daraus ein komplexes Ressourcenmanagementsystem zu machen.

## 8. Ansichten

Ein wichtiges Prinzip lautet:

**Ansichten referenzieren vorhandene Objekte, statt parallele Aufgabenbestände anzulegen.**

Das betrifft unter anderem:

- Listenansicht
- Tabellenansicht
- Reiter
- Pinnwand
- In Bearbeitung
- Verspätet
- Labels
- gespeicherte Filter
- Mein Tag
- Tagesplanung

### Reiter

Aufgabe, Long-Task oder Gruppe können als Reiter geöffnet werden.

Maximal zwölf Punktreiter bleiben gleichzeitig geöffnet; beim dreizehnten wird der am längsten unbenutzte geschlossen.

### Pinnwand

Punkte können in Listen- oder Ordnerpinnwänden als Karten dargestellt werden.

Dabei bleiben sie dieselben Aufgabenobjekte. Bis zu 500 Karten je Pinnwand sind dokumentiert.

### Tabellenansicht

Die Tabelle bildet verschachtelte Aufgaben flach ab und besitzt listenspezifische Spalten.

### Gespeicherte Filter

Filter sind gespeicherte Ansichtsdefinitionen, keine Datenkopien.

Dieses Konzept ist konsistent und sollte auch bei zukünftigen Funktionen beibehalten werden.

## 9. Erinnerungen und Benachrichtigungen

Glide besitzt seit 3.8 lokale Erinnerungen.

Möglich sind unter anderem:

- zur Fälligkeit,
- 10 Minuten vorher,
- eine Stunde vorher,
- einen Tag vorher,
- eigener Minutenabstand,
- fester einmaliger Zeitpunkt.

Aufgaben und Long-Tasks unterstützen Erinnerungen; Gruppen und Überschriften nicht.

Die Anwendung prüft im laufenden Zustand alle 15 Sekunden.

## Stufe A – vorhanden

Wenn Glide läuft, aber verdeckt oder minimiert ist, kann Taskleiste beziehungsweise Dock Aufmerksamkeit anfordern, ohne den Fokus zu stehlen.

## Stufe B – geplant

Eine echte native Systembenachrichtigung unter dem Namen Glide ist an Packaging gebunden.

Dazu werden unter Windows eine stabile AppUserModelID und unter macOS ein echtes .app-Bundle mit Bundle-Identifizier benötigt.

## Stufe C – bewusst nicht umgesetzt

Ein separater Hintergrund-/Autostartprozess für Meldungen bei vollständig geschlossener App ist derzeit ausdrücklich ausgeschlossen.

Grund ist nicht fehlendes Know-how, sondern das Risiko eines zweiten Prozesses auf derselben lokalen beziehungsweise extern synchronisierten Datenablage. Datenintegrität hat hier bewusst Vorrang.

Diese Architektur sollte im Marketing korrekt dargestellt werden.

# 10. Vorlagen und Workflows

Die Vorlagenfunktion ist inzwischen ein wesentlicher Bestandteil und nicht nur ein Nebenfeature.

Aktuell existieren **16 Praxisvorlagen**.

## Listenbasierte Vorlagen

Unter anderem:

- Tagesplanung
- Projektstart
- Einkauf
- Wochenplanung
- Besprechung
- Reise
- Einarbeitung
- Veröffentlichung
- Haushalt
- Rückblick

## Ordner-/Projektvorlagen

Unter anderem:

- Projekt
- Veranstaltung
- Immobilienvermarktung

- WordPress-Relaunch
- WEG-Verwaltung
- Baukommunikation

Die Vorlage „Einarbeitung“ enthält beispielsweise Aufgaben wie Arbeitsplatz und Zugänge vorbereiten und beschreibt damit das **Onboarding einer Person**, nicht das Onboarding in Glide selbst.

Der Vorlagenkatalog unterstützt deutlich mehr als leere Checklisten:

- Beschreibungen,
- Labels,
- Phasen,
- Gruppen,
- Überschriften,
- Unterpunkte,
- Long-Tasks,
- Fristen,
- teilweise Bearbeitungstage,
- Aufwände,
- Wiederholungen,
- Anhänge,
- verschachtelte Ordnerstrukturen.

Das macht die Vorlagen zu einem echten **Workflow-Starter-System**.

## 11. Note-Taking-Fähigkeiten

Glide besitzt inzwischen durchaus Notizfunktionen, ist aber **kein vollständiges PKM- oder Note-Taking-System**.

### Vorhanden

Notizkontext kann über:

- Aufgabenbeschreibungen,
- Long-Tasks,
- Listenbeschreibungen,
- Ordnerbeschreibungen,
- Anhänge,
- Besprechungsvorlagen,
- Reiter,
- Pinnwände

abgebildet werden.

Das reicht gut für:

- Briefings,
- kurze Protokolle,
- Arbeitsnotizen,
- Entscheidungsnotizen,
- Aufgabenhintergrund,
- projektnahes Referenzmaterial.

### Nicht vorhanden

Im vorliegenden Produktmodell fehlen bewusst beziehungsweise derzeit:

- echter Rich-Text-Editor,
- frei verlinkte Wissensseiten,
- Backlinks,
- Knowledge Graph,
- Block-Editor,

- eingebettete Datenbanken,
- Web Clipper,
- OCR-/Dokumentsuche,
- umfangreiche Mediennotizen,
- Notizversionierung,
- Plugin-Ökosystem.

## Einordnung

Glide sollte daher nicht als „Notion-/Obsidian-Ersatz“ positioniert werden.

Besser:

**Task-Manager mit starkem Arbeitskontext und leichten Notizfunktionen.**

## 12. Datenhaltung

Die Nutzdaten sind vom Programm getrennt.

| Bestandteil         | Zweck                                                      |
|---------------------|------------------------------------------------------------|
| liste_speicher.json | Listen, Ordner, Aufgaben, Labels, Papierkorb, Verlauf usw. |
| settings.json       | persönliche Einstellungen und UI-Zustände                  |
| vorlagen.json       | eigener Vorlagenkatalog                                    |
| attachments/        | lokale Anhänge                                             |
| backups/            | Sicherungen                                                |
| glide.lock          | lokale Belegungssperre                                     |
| datenordner.json    | gerätebezogener Ablagezeiger                               |

Plattformabhängig wird ein Benutzer-Datenordner verwendet; für Tests überschreibt `GLIDE_DATA_DIR` den Pfad.

Glide greift für seine Kernfunktionen weder auf ein Benutzerkonto noch auf einen eigenen Server zu.

## 13. Datenformat 15

Das aktuelle Aufgabenformat ist **15**.

Die Evolutionslogik ist bemerkenswert konservativ.

### Format 14

3.14 ergänzte:

- `planned_date`
- `estimated_minutes`

### Format 15

3.19 ergänzte den dauerhaften Änderungsverlauf.

Ein vereinfachtes Modell sieht konzeptionell so aus:

Glide-Datenbestand

```

├─ version = 15
├─ folders
└─ lists

```

```
|  └─ items
|  └─ labels
|  └─ trash
|  └─ active references
|  └─ history
```

Ein Aufgabenelement kann unter anderem enthalten:

text  
done  
importance  
due  
due\_time  
repeat  
reminder  
planned\_date  
estimated\_minutes  
kind  
color  
labels  
description  
attachments  
children

Der Verlauf liegt getrennt daneben.

## Migrationsprinzip

Vor einem Formatsprung wird eine unveränderte Originalkopie angelegt.

Beispielsweise:

liste\_vor\_format15\_<Zeitstempel>.json

Scheitert diese Sicherung, wird der ältere Bestand nicht überschrieben.

## 14. Änderungsverlauf

Das Datenformat 15 besitzt einen persistenten Verlauf mit maximal 4.000 Einträgen.

Er kann unter anderem Vorgänge wie:

- angelegt,
- geändert,
- erledigt,
- wieder geöffnet,
- verschoben,
- umbenannt,
- Papierkorb,
- Wiederherstellung,
- endgültig entfernt

erfassen.

Wichtig:

Der Verlauf ist **kein vollständiges Version-Control-System**.

Er speichert beispielsweise nicht die alten vollständigen Feldinhalte und bietet keine Wiederherstellung einer früheren Punktversion.

Undo und Verlauf sind bewusst getrennt. Ein Undo verändert den Bestand erneut und erzeugt entsprechend einen weiteren Verlaufsvorgang.

## 15. Automatisches Speichern und lokale Sicherungen

Im Anwendungscode sind mehrere Schutzebenen erkennbar:

- Autosave: **alle 5 Minuten**
- mindestens 10 Sicherungen behalten
- maximal 40 automatische JSON-Sicherungen
- frühestens nach 120 Sekunden erneut sichern
- zeitbezogene Backuprotation

Diese lokale Sicherung ist von den exportierbaren Backups zu unterscheiden.

## 16. Portable Aufgabenbackups

`.glidebackup` ist technisch ein ZIP-Archiv aus:

- `data.json`
- referenzierten Anhängen.

Das System prüft unter anderem Archivpfade, Größen und Datenstruktur.

Dokumentierte Sicherheitsgrenzen umfassen:

- bis 512 MiB je Anhang,
- maximal 2 GiB entpackte Daten,
- maximal 32 MiB Daten-JSON,
- technisches Validierungslimit 200.000 Punkte,
- maximale Punkttiefe 100.

Diese Grenzwerte sind technische Sicherheitsgrenzen und ausdrücklich keine Performancezusage.

## 17. Vollständiges App-Backup

Seit 3.16 gibt es zusätzlich `.glideapp`.

Dieses Archiv erweitert das normale Aufgabenbackup um:

- persönliche Einstellungen,
- Vorlagen,
- Aktivitätsdaten.

Vor einer Wiederherstellung zeigt Glide eine Inhaltsvorschau mit Version, Datenformat und Mengen an. Einzelne Bereiche lassen sich getrennt auswählen.

Vor dem Ersetzen entstehen wiederum Rückfallsicherungen für:

- Aufgaben,
- Einstellungen,
- Vorlagen.

## Grenzen des Backup-Konzepts

Nicht vorhanden sind derzeit:

- automatisches Offsite-Backup,

- Cloudbackup,
- Backup-Zeitplan,
- Zusammenführen zweier vollständiger Bestände,
- Passwortschutz,
- Backupverschlüsselung,
- interne Versionsgeschichte innerhalb eines Archivs.

## Bewertung

Das lokale Backupkonzept ist **für eine Desktop-App bereits ungewöhnlich ausgereift**.

Was fehlt, ist weniger eine weitere lokale Backupfunktion als ein verständliches Konzept für:

**„Was passiert, wenn der gesamte Computer ausfällt?“**

Dafür braucht es entweder ein bewusstes externes Backupziel oder eine klare Nutzeranleitung.

## 18. Extern synchronisierte Datenordner

Glide darf seinen Datenordner in einem extern synchronisierten Ordner verwenden.

Das ist aber ausdrücklich **keine Glide-Synchronisierung**.

Die Lock-Datei kann nicht verhindern, dass auf zwei Geräten jeweils eine noch nicht synchronisierte Kopie gleichzeitig verändert wird.

Der vorgesehene Ablauf ist deshalb:

**Glide schließen → Synchronisierung vollständig abwarten → auf anderem Gerät öffnen.**

Das ist eine wichtige Produktgrenze.

Glide bietet aktuell:

**dateibasierte Portabilität**

statt

**verteilte Synchronisation mit Konfliktauflösung.**

## 19. Import und Export

### TXT und Markdown

Geeignet als lesbare Austauschformate.

Nicht als verlustfreier vollständiger Rückweg gedacht.

### CSV

3.18 bringt einen umfangreichen CSV-Import:

- Erkennung von Trennzeichen,
- Erkennung verschiedener Kodierungen,
- Spaltenzuordnung,
- Vorschau,
- Verschachtelung,
- Rückgängig,
- Import in verschiedene Ziele.

Grenzen:

- kein XLSX,
- keine Anhänge aus einer Spalte,

- keine Erinnerungen/Wiederholungen aus einer Spalte,
- kein Abgleich mit vorhandenen Aufgaben,
- keine gespeicherten Mappingprofile,
- maximal 5.000 Zeilen,
- 64 Spalten,
- 12 MB.

## Drucken/PDF

Glide erzeugt HTML und übergibt den eigentlichen Druck beziehungsweise die PDF-Ausgabe an die Betriebssystem-/Browserfunktion.

Das vermeidet eine eigene PDF-Laufzeitabhängigkeit.

## 20. Kalenderexport und -import

### ICS-Ausgabe

Seit 3.20 kann Glide Kalenderdateien erzeugen.

Es handelt sich ausdrücklich um eine **Dateiausgabe**, nicht um eine Kalenderverbindung.

Kein:

- Konto,
- Abonnement,
- automatischer Rückweg,
- laufender Sync.

Auch Punkte ohne Fälligkeit können bei aktivierter Planungsoption über ihren Bearbeitungstag als Planungstermin ausgegeben werden.

### ICS-Import

3.21 liest eine vom Nutzer gewählte ICS-Datei.

Eigene Glide-UIDs werden beim Rundlauf wiedererkannt und nicht doppelt angelegt. Fremde UIDs werden dagegen nicht dauerhaft gespeichert; dieselbe fremde Kalenderdatei zweimal zu importieren kann entsprechend doppelte Aufgaben erzeugen.

Grenzen:

- maximal 2.000 Termine,
- maximal 12 MB,
- keine Kalender-Synchronisierung,
- keine Teilnehmer,
- keine Anhänge,
- keine EXDATE-Ausnahmen,
- keine VTOD/VJOURNAL-Objekte,
- keine importierten VTIMEZONE-Definitionen.

Das ist eine sinnvolle Grenze für ein lokales Aufgabenprodukt.

## 21. Oberfläche und Bedienkonzept

Glide hat sich stark von einer einfachen Tk-Liste entfernt.

### Zentrale Oberflächenbereiche

Unter anderem:

- Startseite

- Vorlagen
- Listen/Ordner
- In Bearbeitung
- Mein Tag
- Tagesplanung
- Labels
- Verspätet
- Papierkorb
- Kalender

## Personalisierung

Dokumentiert beziehungsweise umgesetzt sind:

- Hell-/Dunkelmodus
- Akzentfarbe
- drei Schriftgrößen
- Startansicht
- Wochenbeginn
- Sekundenanzeige
- Mondphase
- Jahres-/Aktivitätsanzeige
- Materialoptik
- persönlicher Name/Monogramm
- Tagesziel
- Tageskapazität

Die Materialdarstellung ist technisch bewusst kein echter transparenter per-Widget-Blur, sondern arbeitet mit getönten opaken Flächen, Kanten und optional Windows-DWM.

## 22. Onboarding

Hier muss zwischen **Entwickler-Onboarding** und **Endnutzer-Onboarding** unterschieden werden.

### Entwickler-Onboarding: stark

Bereits vorhanden sind:

- 03\_STARTKONTEXT.md
- Architektur
- Projektübergabe
- Datenvertrag
- QA-Bericht
- Dokumentationsindex
- Release-Checkliste

Für die Weiterentwicklung ist das inzwischen gut.

### Endnutzer-Onboarding: noch nicht ausreichend nachgewiesen

Im vorliegenden Bestand sehe ich dagegen **keinen dedizierten First-Run-Onboarding-Ablauf**.

Die Vorlage „Einarbeitung“ ist kein Glide-Tutorial, sondern ein Workflow zum Einarbeiten einer Person.

Vorlagen und Beispieldaten helfen beim Einstieg, ersetzen aber kein Produkt-Onboarding.

### Sinnvoller First-Run-Ablauf

Ein sehr kurzes Glide-Onboarding sollte fünf Dinge erklären:

1. **Ordner → Listen → Gruppen → Aufgaben**

2. **Fälligkeit ≠ Bearbeitungstag ≠ Mein Tag**
3. **Listenansicht, Tabelle, Reiter und Pinnwand zeigen dieselben Aufgaben**
4. **Wo die Daten liegen und wie ein Backup erzeugt wird**
5. **Benachrichtigungen funktionieren nur entsprechend dem jeweiligen Betriebszustand**

Optional sollte anschließend eine Beispielvorlage angeboten werden.

Das wäre für die Produktreife wichtiger als viele weitere neue Funktionen.

## 23. Branding und Produktidentität

Hier ist der Stand zweigeteilt.

### Technische Produktidentität: weit entwickelt

PRODUCT\_IDENTITY.md dokumentiert bereits:

- Produktname
- Version
- Datenformat
- Plattformpfade
- Backupformat
- Grenzen
- Sprache
- Schrift
- Symbole
- technische Identitäten

### Geschäftliche Markenidentität: noch offen

Noch nicht endgültig festgelegt sind unter anderem:

- Publisher/Herausgeber
- Copyright-Zeile
- Datenschutz-URL
- Lizenzmodell
- Preis/Monetarisierung
- Windows-AppUserModelID
- Inno-Setup-AppId
- macOS-Bundle-Identifizier
- macOS-Mindestversion
- Zielarchitekturen
- Sicherheitskontakt
- professionelle Markenprüfung „Glide“

Die Verwendung des Namens **Glide** sollte deshalb vor einer öffentlichen Vermarktung professionell markenrechtlich geprüft werden.

Das ist keine Formalität, die man erst nach Erstellung aller Storegrafiken erledigen sollte.

## 24. Positionierung und Markenversprechen

„Local-first“ alleine ist inzwischen kein ausreichendes Alleinstellungsmerkmal.

Obsidian speichert seine Notizen ebenfalls lokal in normalen Markdown-Dateien und kann vollständig offline genutzt werden. Anytype positioniert sich ebenfalls konsequent local-first und bietet mittlerweile sogar verschlüsselte Synchronisation und Kollaboration.

Glides tatsächliche Differenzierung liegt daher in der **Kombination**.

## Empfehlenswerte Positionierung

**Glide ist ein ruhiger, task-first Desktop-Planer für Menschen, die ihre Arbeit bewusst planen und ihre Daten selbst besitzen wollen – ohne Konto- oder Cloudzwang.**

## Drei Markenpfeiler

### Planen

Fälligkeit, Bearbeitungstag, Mein Tag, Aufwand und Tageskapazität.

### Organisieren

Listen, Ordner, Gruppen, Labels, Filter, Tabelle, Reiter, Pinnwand und Vorlagen.

### Besitzen

Lokale Daten, kontrollierte Migration, portable Backups, keine Telemetrie und keine zwingende Cloud.

Eine mögliche kurze Markenformel wäre:

**Planen. Ordnen. Besitzen.**

Oder etwas beschreibender:

**Plane deinen Tag. Ordne deine Arbeit. Behalte deine Daten.**

Diese Formulierungen sind strategische Vorschläge, keine bereits dokumentierten Claims.

## 25. Aktueller Konkurrenzvergleich – Aufgabenmanagement

### Todoist

Todoist bietet sehr ausgereifte Schnellerfassung per natürlicher Sprache, Wiederholungen, Erinnerungen, Listen-/Kalender-/Boardansichten sowie Zusammenarbeit über Projekte, Zuweisungen und Kommentare. Auch Aufgabendauern und kalenderbasierte Zeitplanung sind inzwischen ausgebaut.

**Todoist ist Glide voraus bei:**

- Cloud-Sync
- Mobile
- Teamfunktionen
- Integrationen
- natürlicher Sprache
- Ökosystem
- Veröffentlichung/Reifegrad

**Glide differenziert sich bei:**

- kein notwendiges Konto
- lokale Datenhoheit
- transparentes Backup-/Migrationsmodell
- Trennung Fälligkeit/Bearbeitungstag/Mein Tag
- Tageskapazität
- Reiter/Pinnwand als alternative Sichten
- kontrollierbarer lokaler Datenordner

### TickTick

TickTick ist funktional noch breiter: natürliche Sprache, Wiederholungs- und Standorterinnerungen, mehrere Kalenderansichten, Liste/Kanban/Timeline, Notizen und Markdown in Aufgaben, Habit Tracker, Pomodoro, Eisenhower-Matrix, Statistiken und Kollaboration.

Glide sollte nicht versuchen, TickTick Feature für Feature nachzubauen.

TickTicks Stärke ist:

**maximale Produktivitätsbreite.**

Glides sinnvollere Stärke ist:

**weniger Systemumfang, dafür lokaler, kontrollierbarer und transparenter.**

## Microsoft To Do

Microsoft To Do konzentriert sich auf Listen, Fälligkeiten, Erinnerungen, Wiederholungen, Schritte und Notizen. „Mein Tag“ wird täglich neu zusammengestellt und Vorschläge helfen bei der Auswahl. Die Aufgaben liegen auf Exchange Online und integrieren sich entsprechend in Outlook.

Microsoft To Do ist deutlich einfacher als Glide.

Glide besitzt bereits:

- komplexere Hierarchien,
- weitere Punktarten,
- Vorlagen,
- Bearbeitungstag,
- Aufwand,
- Tageskapazität,
- Pinnwand,
- Reiter,
- Tabelle,
- lokale Backups.

Microsoft hat dagegen den erheblichen Vorteil des Microsoft-Ökosystems und geräteübergreifender Synchronisation.

## Things

Things ist konzeptionell der **nächste Vergleich zu Glides Planungslogik**.

Things unterscheidet zwischen Start-/When-Datum und Deadline. Aufgaben können bewusst in Today, Upcoming, Anytime oder Someday gesteuert werden.

Das ähnelt stark der Grundidee:

**Bearbeitungstag ≠ Fälligkeit**

in Glide.

Things bietet außerdem Tags, Notizen, Erinnerungen und Natural-Language-Eingaben.

Things ist allerdings ausschließlich auf Apple-Plattformen verfügbar; Windows, Android, Linux und Web werden nicht unterstützt.

## Konsequenz für Glide

Von Things sollte Glide vor allem **konzeptionelle Klarheit und Oberflächenruhe** als Referenz ansehen.

Glide muss Things nicht durch zusätzliche Features schlagen.

Eine Kombination aus:

- ähnlich klarer persönlicher Planung,
- Windows + macOS,
- lokaler Datenhoheit,
- Tabelle,
- Reiter,
- Pinnwand,
- Aufwand/Kapazität

kann ein eigenständiges Profil bilden.

## 26. Aktueller Konkurrenzvergleich – Note-Taking und Wissensmanagement

### Notion

Notion verbindet Dokumente und sehr flexible Datenbanken. Datenbanken verfügen über Eigenschaften, Filter, Sortierungen und beliebig viele Ansichten; Aufgaben-Datenbanken können Status, Zuständige und Fälligkeit verlangen und über „My tasks“ aggregiert werden.

Wichtig: Notion besitzt inzwischen einen echten Offline-Modus. Seiten können offline verfügbar gemacht und bearbeitet werden; auf bestimmten Plänen werden zuletzt verwendete und favorisierte Seiten automatisch bereitgestellt. Datenbank-Offlinezugriff besitzt weiterhin spezifische Grenzen, etwa die ersten 50 Zeilen der ersten Ansicht.

Daher wäre die Aussage „Notion funktioniert nicht offline“ inzwischen falsch.

#### Notion ist Glide überlegen bei:

- Dokumenten
- Rich Text
- Datenbanken
- benutzerdefinierten Eigenschaften
- Relation zwischen Daten
- Teamarbeit
- Automatisierungen
- flexiblem Workspace-Aufbau

#### Glide ist fokussierter bei:

- unmittelbarer persönlicher Aufgabenplanung
- weniger Setup-Komplexität
- lokaler App-Datenablage ohne Accountpflicht
- explizitem Backup-/Migrationsmodell

### Obsidian

Obsidian ist einer der wichtigsten Konkurrenten beim Thema Datenhoheit.

Es verwendet normale lokale Markdown-Dateien und kann offline betrieben werden.

Mit **Bases** kann Obsidian inzwischen sogar datenbankartige Ansichten auf lokale Markdown-Dateien legen. Tabelle, Liste, Karten und weitere Ansichten arbeiten auf denselben lokalen Dateieigenschaften.

Obsidian besitzt zusätzlich ein großes Erweiterungsökosystem.

### Unterschied

Obsidian ist:

**note-first / knowledge-first**

Glide ist:

**task-first / planning-first**

Glide sollte deshalb nicht versuchen, Backlinks, Wissensgraph, Markdown-Wiki und Plugins nachzubauen.

Das würde seine Stärke verwässern.

### Anytype

Anytype ist beim Thema **Local-first + Datenhoheit** der direkte strategische Gegenbeweis dafür, dass Local-first allein ein USP wäre.

Anytype kombiniert lokale Daten, verschlüsselte Synchronisierung und inzwischen local-first Kollaboration. Der zugrunde liegende Sync arbeitet mit On-Device-Speicherung, P2P-Synchronisation und CRDTs.

Das Produktmodell umfasst darüber hinaus Notizen, Aufgaben, Seiten und anpassbare Objekte.

Glide kann deshalb nicht glaubwürdig sagen:

„Wir sind einzigartig, weil die Daten lokal liegen.“

Besser ist:

**Glide verzichtet bewusst auf die Komplexität eines verteilten Wissenssystems und bietet dafür einen überschaubaren, persönlichen Aufgaben- und Planungsworkflow.**

## Evernote

Evernote ist weiterhin klar note-first.

Es verbindet Notizbücher, Suche, Aufgaben, Kalender, Web Clipper, Zusammenarbeit und inzwischen umfangreiche KI-Funktionen.

Glide konkurriert damit nur am Rand.

Evernote ist wesentlich geeigneter für:

- Wissenssammlung,
- Webrecherche,
- Dokumentablage,
- umfangreiche Notizen.

Glide ist geeigneter als Produktkonzept für:

- konkrete Arbeit,
- Planung,
- Tagesfokus,
- lokale Kontrolle.

## 27. Zusammenfassende Wettbewerbsmatrix

| Bereich                  | Glide             | Todoist            | TickTick       | Things                                   | Notion                      | Obsidian                      | Anytype                | Evernote             |
|--------------------------|-------------------|--------------------|----------------|------------------------------------------|-----------------------------|-------------------------------|------------------------|----------------------|
| Aufgabenfokus            | <b>hoch</b>       | hoch               | hoch           | hoch                                     | mittel/<br>hoch             | gering<br>ohne<br>Setup       | mittel                 | mittel               |
| Tagesplanung             | <b>hoch</b>       | hoch               | hoch           | <b>sehr<br/>hoch</b>                     | konfigurierbar              | plugin-/<br>setupabh<br>ängig | konfigurierbar         | mittel               |
| Aufwand/<br>Kapazität    | <b>integriert</b> | Dauer<br>vorhanden | stark          | begrenzt                                 | frei<br>modellierbar        | frei<br>modellierbar          | frei<br>modellierbar   | begrenzt             |
| Notiztiefe               | mittel            | gering/<br>mittel  | mittel         | gering/<br>mittel                        | <b>sehr<br/>hoch</b>        | <b>sehr<br/>hoch</b>          | <b>sehr<br/>hoch</b>   | <b>sehr<br/>hoch</b> |
| Lokale<br>Datenhöhe      | <b>hoch</b>       | cloudzentriert     | cloudzentriert | Cloud<br>optional<br>für Sync            | cloudzentriert +<br>Offline | <b>sehr<br/>hoch</b>          | <b>sehr<br/>hoch</b>   | cloudzentriert       |
| Konto für<br>Kernbetrieb | <b>nein</b>       | ja                 | ja             | lokal<br>möglich,<br>Cloudkonto für Sync | ja                          | nein                          | je nach<br>Betriebsart | ja                   |
| Team/<br>Kollaboration   | nein              | hoch               | vorhanden      | nein                                     | <b>sehr<br/>hoch</b>        | mit<br>Zusatzdiensten         | <b>hoch</b>            | hoch                 |
| Mehrere<br>Ansichten     | <b>hoch</b>       | hoch               | hoch           | mittel                                   | <b>sehr<br/>hoch</b>        | hoch                          | hoch                   | mittel               |

| Bereich          | Glide                                | Todoist       | TickTick      | Things          | Notion | Obsidian        | Anytype     | Evernote |
|------------------|--------------------------------------|---------------|---------------|-----------------|--------|-----------------|-------------|----------|
| Portable Backups | <b>expliziter Produktbestandteil</b> | Serviceexport | Serviceexport | App-Daten/Cloud | Export | normale Dateien | Export/Sync | Export   |
| Mobile           | nein                                 | ja            | ja            | Apple           | ja     | ja              | ja          | ja       |
| Windows          | Zielplattform                        | ja            | ja            | nein            | ja     | ja              | ja          | ja       |

Die Tabelle ist bewusst eine **Produkteinordnung**, keine Rangliste.

## 28. Das tatsächliche Alleinstellungsmerkmal

Nach dem aktuellen Konkurrenzvergleich würde ich das USP deutlich schärfer formulieren.

### Nicht ausreichend

„Glide ist lokal.“

Obsidian und Anytype sind ebenfalls sehr stark in diesem Bereich.

### Nicht ausreichend

„Glide hat viele Ansichten.“

Notion, Obsidian, TickTick und Todoist besitzen ebenfalls unterschiedliche Views.

### Nicht ausreichend

„Glide kann Aufgaben und Notizen.“

Das können fast alle relevanten Produkte.

## Glaubwürdige Kombination

Glide kombiniert:

1. **task-first statt workspace-first**
2. **Fälligkeit, Bearbeitungstag und Mein Tag als getrennte Konzepte**
3. **Aufwand und Tageskapazität**
4. **mehrere Sichten auf dieselben Aufgaben**
5. **umfangreiche Vorlagen**
6. **lokale Daten ohne Kontozwang**
7. **explizite Backup-/Migrationskontrolle**
8. **ruhige Desktopoberfläche**
9. **keine Telemetrie und keine zwingende Cloud**

Daraus ergibt sich ein wesentlich stärkeres Versprechen:

**Ein persönlicher Desktop-Planer, der nicht nur festhält, was erledigt werden muss, sondern auch wann du daran arbeiten willst – und bei dem deine Daten tatsächlich bei dir bleiben.**

## 29. Qualitätssicherung

Die interne QA ist mittlerweile eine der bemerkenswertesten Eigenschaften des Projekts.

Der vollständige Prüfplan umfasst 25 Testsuiten sowie Syntax-, Versions-, Dokument-, Fixture- und Analyseprüfungen und reproduziert zusätzlich Beispiel- und Releasedaten.

Hinzu kommen spezialisierte Werkzeuge für:

- statische Codeanalyse,
- Erreichbarkeitsanalyse,

- Dokumentationsstand,
- Screenshot-Erzeugung,
- Symbol-/Fontprüfung,
- Performance,
- Dauerbetrieb,
- Beispieldaten,
- Releasebestände,
- Vorlagen.

Damit ist Glide intern deutlich besser abgesichert als viele typische Einzelentwicklerprojekte.

## 30. Noch nicht abgeschlossene Qualitätsnachweise

Der grüne Gesamtlauf darf trotzdem nicht mit einer fertigen Desktopveröffentlichung verwechselt werden.

Die Release-Checkliste nennt weiterhin ausdrücklich:

- native Windows-Prüfung,
- echte macOS-/Windows-Eingabegeräte,
- DPI und mehrere Monitore,
- Screenreader,
- Hochkontrast/RDP,
- physisches Schlafen/Aufwachen,
- Dock-/Taskleistenwirkung,
- Dauerlauf,
- real synchronisierte Datenordner.

### Dauerlauf

Das Werkzeug existiert.

Ein belastbarer Lauf benötigt mindestens zehn Minuten und standardmäßig 4.000 synthetische Aufgaben.

Ein Ein-Minuten-Lauf mit 500 Aufgaben gilt ausdrücklich nur als Smoke-Test.

Ein aktuelles Ergebnis eines solchen vollständigen Dauerlaufs liegt mir im vorliegenden Material nicht vor.

### Performance

Das Performancewerkzeug misst:

- Startzeit,
- Rendering,
- Speichern,

bei:

- 100,
- 1.000,
- 5.000 Aufgaben,

jeweils mit und ohne Materialmodus.

Es erklärt selbst ausdrücklich, dass es kein Langzeit-, Speicher- oder Mehrmonitorbenchmark ist.

Ein aktueller Ergebnisbericht wurde mir ebenfalls nicht vorgelegt.

## 31. Architektur und technische Schulden

Die Architekturdatei bezeichnet `app.pyw` selbst ausdrücklich als **kanonischen Python-/Tk-Monolithen**.

Das ist inzwischen der größte technische Risikobereich.

Die statische Analyse nennt für den Kern:

- 26.736 Zeilen,
- 953 Funktionen/Methoden,
- 253 Konstanten,
- 53 Funktionen mit mehr als 80 Zeilen.

Besonders groß sind beispielsweise:

- `item_form_dialog`: 596 Zeilen
- `create_ui`: 518
- `open_calendar_view`: 363
- `refresh_home`: 289
- `show_settings_dialog`: 265
- `validate_backup_schema`: 241

Zusätzlich findet die Analyse:

- Verschachtelung bis 11 Ebenen,
- 132 Fundstellen wiederholter Codeblöcke,
- 38 breite `except Exception`,
- aber keine TODO/FIXME/HACK-Markierungen.

Die Erreichbarkeitsanalyse findet von 896 untersuchten Methoden 826 vom Einstieg aus erreichbar sowie 63 Kandidaten, die weder vom Startpfad noch aus den berücksichtigten Tests erreicht werden. Wegen dynamischer Tk-Bindungen ist das **kein Beweis für toten Code**, sondern eine Prüfliste.

## Konsequenz

Der nächste technische Ausbau sollte nicht sofort weitere große Funktionspakete in denselben Monolithen schreiben.

Sinnvoll ist stattdessen eine **stufenweise Konsolidierung**.

Kein Rewrite.

Keine gleichzeitige Änderung des Datenmodells.

Zunächst besonders geeignete Extraktionsbereiche:

- Backup/Migration
- Import/Export
- Änderungsverlauf
- ICS
- große Dialoge
- gemeinsame UI-Helfer
- redundante Schema-Backupfunktionen

Die breite Testsuite macht diese Refaktorisierung inzwischen wesentlich sicherer.

## 32. Umgesetzt, geplant, bewusst ausgeschlossen

### Umgesetzt

Unter anderem:

- Kern-Aufgabenmanagement
- verschachtelte Struktur
- Long-Tasks
- Gruppen/Überschriften
- Anhänge
- Labels

- Wiederholungen
- lokale Erinnerungen
- Dock-/Taskleistenaufmerksamkeit
- Reiter
- Pinnwand
- Schnellerfassung
- gespeicherte Filter
- Mein Tag
- Tabellenansicht
- Bearbeitungstag
- Aufwand
- Tageskapazität
- Tagesplanung
- vollständiges App-Backup
- Druck/PDF
- CSV-Import
- Änderungsverlauf
- ICS-Ausgabe
- ICS-Import
- 16 Praxisvorlagen
- umfangreiche Personalisierung

## Konkrete zukünftige Produktideen

In der aktuellen Projektübergabe werden insbesondere genannt:

### Benutzerdefinierte Felder je Liste

Dies würde voraussichtlich Datenformat 16 erfordern.

### Eigene Ansichten auf Basis dieser Felder

Diese bauen auf den benutzerdefinierten Feldern auf.

### Größere Pinnwandfunktionen

Sind separat zu planen.

## Geplant, aber an Release-Infrastruktur gebunden

### Native Systembenachrichtigungen

Stufe B gemeinsam mit Packaging.

### Bewusst derzeit ausgeschlossen

- eigener Cloudsync
- verteilte Konfliktauflösung
- Mehrbenutzerbetrieb
- Rich-Text-Editor
- Mehrsprachigkeit
- Mailintegration
- echte Kalendersynchronisierung
- Hilfsprozess für geschlossene App
- Telemetrie

Diese Kategorie darf nicht mit „vergessenen Features“ vermischt werden.

## 33. Offene Entscheidungen

### Produkt und Geschäft

| Entscheidung                 | Status                        |
|------------------------------|-------------------------------|
| Publisher/Herausgeber        | offen                         |
| Copyright                    | offen                         |
| endgültige Supportstruktur   | nicht vollständig finalisiert |
| Datenschutz-URL              | offen                         |
| Lizenzmodell                 | offen                         |
| Preis                        | offen                         |
| Monetarisierung              | offen                         |
| Vertriebskanäle              | noch freizugeben              |
| professionelle Markenprüfung | offen                         |

### Windows

- Zielarchitektur
- AppUserModelID
- Inno-Setup-Appld
- Installer
- Signierung

### macOS

- Bundle-Identifizier
- Mindestversion
- Architektur
- Signierung
- Notarisierung

## 34. Fehlende Unterlagen beziehungsweise noch nicht nachgewiesene Ergebnisse

Diese Liste ist gegenüber meinem ursprünglichen Bericht deutlich kürzer geworden.

### Nicht mehr als fehlend anzusehen

Vorhanden sind inzwischen nachweislich:

- aktueller QA-Bericht
- QA-Testplan
- Architekturübersicht
- Produktgrenzen
- Daten-/Migrationsvertrag
- Dokumentationsindex
- Projektübergabe
- Release-Checkliste
- Produktidentitätsregister
- Systembenachrichtigungsentscheidung
- reproduzierbare Vorlagen
- Beispielbestand
- Release-Arbeitsbestand
- Codeanalyse

## Im vorliegenden Material noch nicht als abgeschlossen nachgewiesen

### Technische Nachweise

- aktueller nativer Windows-3.21.4-Gesamtlauf
- manuelle visuelle macOS-Abnahme
- manuelle visuelle Windows-Abnahme
- aktueller vollständiger Dauerlaufbericht
- aktueller Performance-Baselinebericht
- Screenreader-Abnahme
- Hochkontrast-Abnahme
- DPI-/Multi-Monitor-Testmatrix
- Sleep/Wake-Praxistest
- Clean-Machine-Installationsprüfung

### Packaging

- fertiger Windows-Installer
- fertiges macOS-App-Bundle
- reproduzierbare finale Packaging-Anleitung
- Signierungsnachweis
- Notarisierungsnachweis
- Upgrade-/Uninstall-Nachweis

### Produkt-/Rechtsunterlagen

- endgültige Datenschutzseite/-URL
- endgültiges Lizenzmodell
- Preis-/Monetarisierungsentscheidung
- endgültige Publisher-/Copyrightangaben
- Sicherheitskontakt
- Markenfreigabe

### Endnutzer

Im mir vorliegenden Dateisatz ist kein klarer abgeschlossener Nachweis für:

- First-Run-Onboarding
- kompakte Nutzer-Schnellstartanleitung
- nutzerorientierte Backup-/Recovery-Anleitung
- konsolidiertes Support-/Troubleshooting-Handbuch

erkennbar.

Es können innerhalb des vollständigen Projektarchivs weitere Dokumente existieren; der Dokumentationsindex verweist auf einen deutlich größeren Bestand. Die Aussage lautet daher bewusst „**im vorliegenden Dateisatz nicht nachgewiesen**“, nicht pauschal „existiert nicht“.

### Branding

Ebenso sehe ich ein technisches Identitätsregister und umfangreiche Release-/Grafikaufgaben, aber kein eindeutig aktuelles, abgeschlossenes **visuelles Brand-Guideline-Dokument** mit beispielsweise:

- Logo-Regeln
- Safe Space
- Farbsystem
- Typografiehierarchie
- App-Icon-Regeln
- Screenshot-Stil
- Tonalität
- Marketing-Claim.

## 35. Möglichkeiten

Glide besitzt mehrere realistische Entwicklungschancen.

### 1. Eine Nische zwischen Things und Obsidian

Things zeigt, wie klar persönliche Planung sein kann.

Obsidian zeigt, wie stark lokale Datenhoheit als Produktwert sein kann.

Glide kann dazwischen liegen:

**planungsorientierter als Obsidian, datenautonomer als Things.**

### 2. Windows + macOS als Chance

Things ist Apple-only.

Ein vergleichbar ruhiger persönlicher Planer mit Windows-Unterstützung und lokaler Datenhoheit besitzt damit ein nachvollziehbares Profil.

### 3. Datenschutz und kleine Unternehmen

Die lokale Architektur kann für Anwender interessant sein, die Aufgaben und Projektkontext nicht zwingend in einen SaaS-Dienst legen wollen.

### 4. Branchenvorlagen

Vorlagen wie:

- Immobilienvermarktung,
- WEG-Verwaltung,
- Baukommunikation,
- WordPress-Relaunch

zeigen bereits, dass Glide stärker als allgemeine To-do-Apps mit **realen Arbeitsabläufen** starten kann.

### 5. Einfache Datenportabilität als Vertrauensargument

Backup, lokale Ablage und klare Migration lassen sich gut verständlich kommunizieren.

## 36. Herausforderungen

### Produktseitig

Die größte Gefahr ist Feature-Drift.

Wenn Glide gleichzeitig versucht:

- TickTick bei Produktivitätsfeatures,
- Notion bei Datenbanken,
- Obsidian bei Wissen,
- Anytype bei Local-first,
- Things bei UX

zu kopieren, verliert es seine klare Produktidee.

### Technisch

Der Monolith wächst.

Weitere große Features erhöhen die Änderungsrisiken.

## UX

Die Anzahl der Konzepte ist inzwischen erheblich:

- Ordner
- Listen
- Gruppen
- Überschriften
- Aufgaben
- Long-Tasks
- Fälligkeit
- Bearbeitungstag
- Mein Tag
- Reiter
- Pinnwand
- Filter
- Tabelle

Ohne Onboarding kann die funktionale Stärke zur Einstiegshürde werden.

## Release

Ein grüner Python-Quelltest ist noch kein installierbares Produkt.

## Branding

Der Name muss geklärt werden, bevor erhebliche Marketinginvestitionen entstehen.

## 37. Priorisierte nächste Aufgaben

### P0 – Releasefähigkeit nachweisen

#### 1. Windows vollständig prüfen

Aktuellen 3.21.4-Vollrun auf realem Windows ausführen und archivieren.

#### 2. Native visuelle Abnahme

macOS und Windows vollständig durchgehen:

- Layout
- Fokus
- Drag-and-drop
- Dialoge
- Schriften/Symbole
- Dock/Taskbar
- kleine Fenster
- DPI
- Monitorwechsel

#### 3. Dauerlauf

Mindestens dokumentierten 10-Minuten-/4.000-Punkte-Lauf durchführen und Ergebnis archivieren.

#### 4. Performancebaseline

100 / 1.000 / 5.000 Punkte auf definierten Referenzgeräten messen.

#### 5. Accessibility

Screenreader, Tastaturnavigation, Hochkontrast und Fokusführung prüfen.

## 38. P0/P1 – Packaging

Danach:

1. Publisher festlegen
2. Markenfrage klären
3. Plattformidentitäten vergeben
4. Windows Installer bauen
5. macOS .app bauen
6. signieren
7. notarierten Mac-Build erzeugen
8. Clean-Machine-Tests
9. Upgrade/Uninstall testen
10. erst dann native Systembenachrichtigungen Stufe B integrieren

## 39. P1 – Produktreife

### Endnutzer-Onboarding

Vor weiteren großen Features.

### Quick Start / Hilfe

Besonders:

- Datenordner
- Backup
- Restore
- Gruppen vs. Ordner
- Fälligkeit vs. Planung
- Benachrichtigungsgrenzen

### Kleiner Pilot

Die Releaseplanung schlägt selbst einen begleiteten Pilottest mit ungefähr fünf bis acht geeigneten Einzelanwendern auf Windows und Mac vor.

Das wäre inzwischen wertvoller als weitere interne Features.

## 40. P2 – Technische Konsolidierung

Nach stabiler Releasebaseline:

- Modulgrenzen definieren
- Import/Export extrahieren
- Backup/Migration extrahieren
- History extrahieren
- große Dialoge zerlegen
- wiederkehrende UI-Helfer zusammenführen
- Erreichbarkeitskandidaten klassifizieren
- breit gefasste Exception-Behandlung überprüfen
- Duplikate abbauen

Dabei:

**kein Big-Bang-Rewrite.**

Die vorhandenen Tests sollten schrittweise jede Extraktion absichern.

## 41. P2/P3 – Neue Funktionen

Erst anschließend sollte entschieden werden, ob:

### Benutzerdefinierte Felder / Format 16

tatsächlich der nächste große Produktschritt werden.

Das könnte Glide erheblich flexibler machen.

Gleichzeitig bringt es das Produkt deutlich näher an Notion/Obsidian/Anytype und erhöht Komplexität.

Daher sollte vorher die strategische Frage beantwortet werden:

**Soll Glide ein fokussierter persönlicher Planer bleiben oder zu einem konfigurierbaren Arbeitsdaten-System werden?**

Meine Produktempfehlung ist:

**task-first bleiben.**

Benutzerdefinierte Felder nur einführen, wenn sie konkrete Aufgaben-/Planungsworkflows verbessern – nicht als Einstieg in einen allgemeinen Notion-Klon.

## 42. Gesamtbewertung

Mit allen inzwischen vorliegenden Dateien ist Glide wesentlich reifer einzuschätzen als nach der ersten Projektübergabe.

### Funktional

**weit fortgeschritten**

Die Kernfunktion ist nicht mehr das Problem.

### Datenmodell

**klar und konservativ entwickelt**

Formatsprünge sind abgesichert; Migration und Backup besitzen definierte Verträge.

### Backup

**stark**

Mehrere lokale Sicherungsebenen, portable Archive, vollständiges App-Backup und Restore-Vorschau sind vorhanden.

### Dokumentation

**sehr stark für einen internen Entwicklungsstand**

Aktuelle und historische Dokumente werden sauberer getrennt, und Teile der Dokumentationskonsistenz werden inzwischen automatisiert geprüft.

### QA

**ungewöhnlich systematisch**

25 Testsuiten, drei Analysen und reproduzierbare Bestände bilden eine gute Basis.

### Architektur

**funktional, aber wartungsintensiv**

Der Monolith ist das größte technische Langzeitrisiko.

## UI

**umfangreich und eigenständig**

Aber native Plattformabnahme bleibt erforderlich.

## Onboarding

**Entwicklerseitig gut, endnutzerseitig noch klar verbesserungswürdig.**

## Branding

**technische Identität vorhanden; kommerzielle und rechtliche Markenidentität noch nicht abgeschlossen.**

## Note-Taking

**guter Aufgabenkontext, aber kein PKM-System.**

Das sollte so bleiben, solange kein bewusster Strategiewechsel beschlossen wird.

## Marktposition

Glide sollte **nicht über maximale Featurezahl** konkurrieren.

Sein sinnvollstes Profil ist:

**ruhige persönliche Planung + visuelle Arbeitsorganisation + lokale Datenhoheit.**

## 43. Schlussfolgerung

Glide hat inzwischen eine belastbare eigene Produktidee.

Es ist nicht einfach eine weitere To-do-Liste und inzwischen auch nicht mehr sinnvoll als reines Hobby-Skript zu beschreiben.

Der Kern besteht aus einer interessanten Kombination:

**Deadline, Planungstag und Tagesfokus werden bewusst getrennt; Aufgaben können in mehreren Arbeitsansichten erscheinen, ohne dupliziert zu werden; Daten und Backups bleiben transparent lokal kontrollierbar.**

Die nächste Entwicklungsphase sollte deshalb nicht darin bestehen, möglichst schnell noch zehn weitere Features hinzuzufügen.

Die sinnvollste Reihenfolge lautet:

**prüfen → paketieren → onboarden → pilotieren → konsolidieren → erst danach erweitern.**

Wenn diese Schritte sauber abgeschlossen werden, ist der nächste Engpass nicht mehr die Technik, sondern die Produktentscheidung:

**Wie schmal und klar soll Glide langfristig bleiben?**

Genau darin liegt zugleich seine größte Chance.